Teleoperación de un Robot Móvil en un Entorno Virtual Inmersivo con Control Háptico Inalámbrico basado en ESP32

Daniel Peña
Programa de Ingeniería Mecatrónica
Universidad Militar Nueva Granada
Bogotá D.C., Colombia

Johan Martínez
Programa de Ingeniería Mecatrónica
Universidad Militar Nueva Granada
Bogotá D.C., Colombia

Cristian Quintero
Programa de Ingeniería Mecatrónica
Universidad Militar Nueva Granada
Bogotá D.C., Colombia

Docente: Ing. Oscar Caldas — Asignatura: Realidad Virtual (IX semestre) — Laboratorio de Realidad Virtual
Octubre de 2026

***Resumen*—Este artículo presenta el diseño, la implementación y la evaluación de un sistema de teleoperación inmersivo en el que un robot móvil virtual de tracción diferencial, desarrollado en Unity 6, se controla con un dispositivo háptico inalámbrico basado en un microcontrolador ESP32. El control integra una unidad de medición inercial MPU6050, cuya inclinación (pitch y roll) orienta una cámara que sigue al robot; un joystick analógico que comanda las velocidades lineal y angular; un pulsador que cierra la garra para manipular objetos, y un motor vibrador controlado por PWM que reproduce tres patrones diferenciados al detectar colisiones con objetos manipulables, obstáculos o límites. La comunicación con el computador se realiza por Bluetooth SPP a 50 Hz, y un teléfono inteligente funciona como gafas de realidad virtual tipo Cardboard mediante una transmisión estéreo por Wi-Fi con pre-distorsión de lentes. En las pruebas se midió una latencia de ida y vuelta con mediana de 2.9 ms por USB y 46.1 ms por Bluetooth, una constante de tiempo del control de velocidad de 205 ms y un bloqueo físico efectivo en 15 de 15 pruebas de colisión.**

***Abstract*—This paper presents an immersive teleoperation system in which a virtual differential-drive mobile robot built in Unity 6 is controlled by a wireless ESP32-based haptic device that combines an MPU6050 IMU (camera orientation), an analog joystick (robot motion), a gripper button and a PWM vibrotactile actuator with collision-dependent patterns. The device communicates over Bluetooth SPP at 50 Hz, and a smartphone acts as a Cardboard-style head-mounted display through a Wi-Fi stereo stream.**

***Palabras clave*—realidad virtual, teleoperación, háptica vibrotáctil, ESP32, MPU6050, Bluetooth, Unity, robot móvil.**

I. Introducción

La integración entre realidad virtual (RV), teleoperación y sistemas hápticos es un eje fundamental en aplicaciones avanzadas de ingeniería como la robótica médica, la exploración remota, la manufactura inteligente y el entrenamiento basado en simulación [1]–[3]. La teleoperación permite que un operador controle un sistema remoto —real o virtual— mediante dispositivos de entrada que pueden incorporar retroalimentación sensorial, lo que mejora la percepción de los eventos del entorno y la sensación de control [4].

En esta práctica se desarrolló un sistema completo que integra: 1) un entorno virtual inmersivo en Unity 3D con temática de base de exploración en Marte; 2) un robot móvil virtual teleoperado con garra; 3) un dispositivo mecatrónico háptico inalámbrico basado en ESP32; 4) comunicación Bluetooth con el computador, y 5) retroalimentación vibrotáctil diferenciada por tipo de colisión. Como medio de visualización se empleó un iPhone dentro de un visor tipo Cardboard, alimentado por una transmisión estéreo desde Unity por Wi-Fi.

El documento describe el procedimiento paso a paso según las fases de la guía de laboratorio (diseño conceptual, entorno virtual, control háptico, manufactura e integración), los códigos desarrollados, los resultados experimentales y su análisis.

II. Objetivos

A. Objetivo general

Diseñar e implementar un sistema de teleoperación inmersivo en realidad virtual que permita controlar un robot móvil mediante un dispositivo háptico inalámbrico desarrollado por los estudiantes.

B. Objetivos específicos

1) Diseñar en Unity 3D un entorno virtual inmersivo con navegación, límites físicos, obstáculos, robot móvil funcional y sistema de manipulación de objetos.
2) Diseñar y construir un dispositivo inalámbrico con ESP32, IMU, pulsador de agarre, joystick, actuador vibrotáctil por PWM y batería integrada.
3) Implementar la comunicación Bluetooth y el mapeo de la IMU y del joystick hacia las variables de la cámara y del robot, garantizando estabilidad.
4) Validar el sistema con pruebas de implementación cuantitativas (latencia, colisiones, respuesta dinámica y uso).

III. Marco teórico

A. Teleoperación y realidad virtual

Un sistema de teleoperación se compone de un lado maestro (operador y dispositivo de entrada), un canal de comunicación y un lado esclavo (robot). Cuando el esclavo es virtual, la RV permite entrenar y evaluar la interacción sin riesgo [1], [4]. La latencia del canal y la calidad del mapeo entre las variables del

operador y las del robot determinan la estabilidad y la sensación de control [5].

B. Estimación de la inclinación con IMU

La MPU6050 integra un acelerómetro y un giroscopio de tres ejes [6]. El acelerómetro entrega una medida absoluta pero ruidosa de la inclinación respecto a la gravedad:

$$\theta_a = atan2(-a_x, \sqrt{(a_y^2 + a_z^2)}), \quad \varphi_a = atan2(a_y, a_z) \qquad (1)$$

El giroscopio entrega la velocidad angular, que integrada es suave pero deriva en el tiempo. El filtro complementario combina ambas fuentes con un factor α [7]:

$$\theta_k = \alpha\,(\theta_{k-1} + \omega_y\,\Delta t) + (1-\alpha)\,\theta_a\,, \quad \alpha = 0{,}98 \qquad (2)$$

C. PWM y percepción vibrotáctil

La intensidad de un motor vibrador de masa excéntrica se regula con modulación por ancho de pulso (PWM). La percepción diferenciada de vibraciones depende de la amplitud (ciclo útil), la duración de cada pulso y el ritmo de la secuencia [8], por lo que se diseñaron patrones que difieren en esas tres variables.

D. Bluetooth SPP y estereoscopía

El perfil de puerto serie (SPP) de Bluetooth Clásico emula un enlace RS-232 sobre RFCOMM, y Windows lo expone como un puerto COM [9]. Para la visualización estereoscópica se renderiza una imagen por ojo con una separación interpupilar (IPD) de 64 mm y se aplica una pre-distorsión de barril que compensa la distorsión de cojín de los lentes del visor [10]:

$$p' = p\,(1 + k_1\,r^2 + k_2\,r^4)\,/\,z \qquad (3)$$

IV. Materiales y Equipos

TABLA I
MATERIALES, SOFTWARE Y EQUIPOS UTILIZADOS

Elemento	Cant.	Función
ESP32 DevKit V1 (CP2102)	1	Microcontrolador con Bluetooth
MPU6050 (GY-521)	1	IMU: pitch y roll
Joystick KY-023	1	Mover el robot y botón de garra
Motor vibrador DC 3 V	1	Actuador vibrotáctil
Transistor 2N2222, R 1 kΩ, diodo 1N4148	1 c/u	Driver del motor
Batería LiPo 3,7 V + TP4056 + MT3608	1	Alimentación inalámbrica
Filamento PLA	≈ 60 g	Carcasa (manufactura aditiva)
Computador con Windows 11 y Bluetooth	1	Ejecuta Unity
iPhone + visor tipo Cardboard	1	Gafas de RV
Unity 6 (6000.6.2f1)	—	Entorno virtual y física
Arduino IDE 2 + núcleo ESP32 3.3.11	—	Programación del ESP32
SOLIDWORKS 2026	—	Diseño CAD de la carcasa

V. Fase 1 – Diseño Conceptual

A. Arquitectura general del sistema

La Fig. 1 muestra la arquitectura en bloques. El control háptico adquiere la IMU por I²C y el joystick por el ADC, filtra los ángulos y envía una trama de texto a 50 Hz. En el computador, el componente `HapticLink` recibe las tramas, mide la latencia y reenvía los comandos de vibración; `RobotController` y `RobotGripper` gobiernan el robot; `ChaseCamera` orienta la cámara con la IMU; el motor de física detecta colisiones y `PhoneStreamer` envía la vista estéreo al celular.

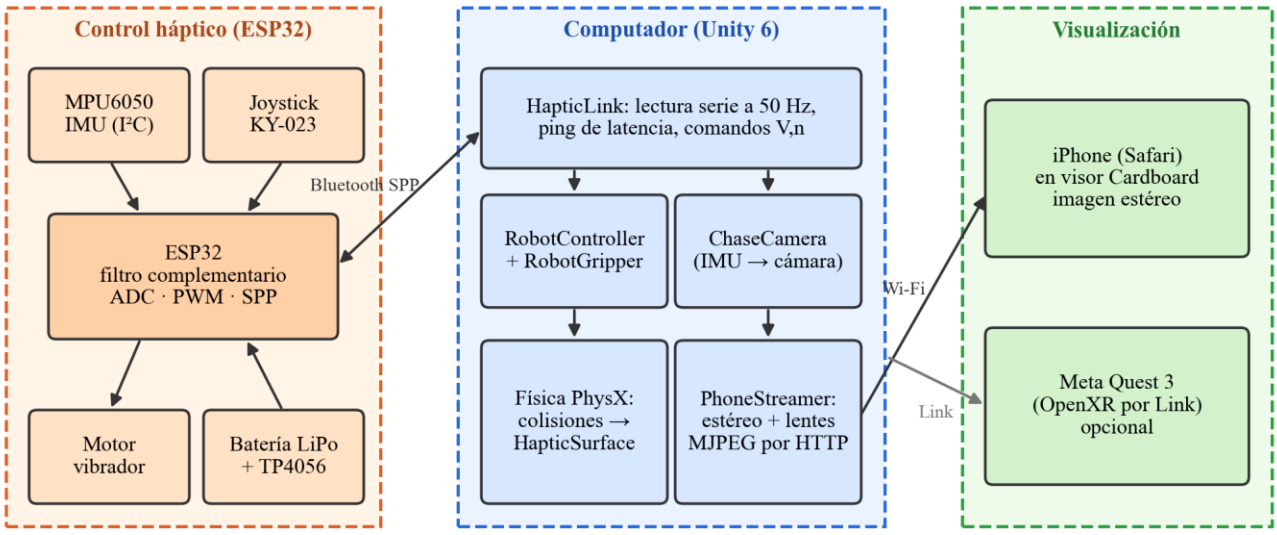


Fig. 1. Arquitectura del sistema: control háptico ESP32, computador con Unity y dispositivos de visualización.

B. Mapeo de variables

Se definió el siguiente mapeo entre el operador y el entorno virtual (Tabla II): el joystick controla el robot (eje Y → velocidad lineal, eje X → velocidad angular), la inclinación de la IMU controla la cámara que sigue al robot (pitch → elevación, roll → giro alrededor del robot) y el pulsador del joystick cierra y abre la garra. Se usó el joystick para el robot y la IMU para la cámara porque así la cámara cumple el papel de la cabeza del operador, mientras que el desplazamiento del robot se comanda con el pulgar sin cambiar el punto de vista.

TABLA II
MAPEO DE VARIABLES DEL CONTROL

Entrada	Variable virtual	Rango
Joystick Y (−1…1)	Velocidad lineal v	±v_max
Joystick X (−1…1)	Velocidad angular ω	±90 °/s
Botón del joystick	Garra (cerrar/abrir)	0/1
Pitch IMU	Elevación de la cámara	3°…80°
Roll IMU	Azimut alrededor del robot	±150°
Colisión (Unity)	Patrón de vibración	1, 2, 3

C. Comunicación

La Fig. 2 resume el intercambio de mensajes. El ESP32 envía `D,pitch,roll,jx,jy,b` cada 20 ms; Unity envía un ping `P,id` cada segundo (respuesta `K,id`) para medir la latencia y comandos `V,n` al detectar colisiones. El celular solicita el video por HTTP.

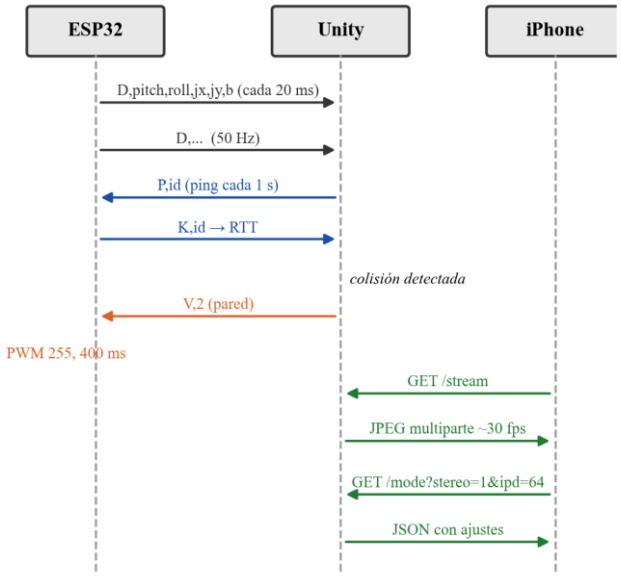


Fig. 2. Diagrama de comunicación entre el ESP32, Unity y el celular.

TABLA III
PROTOCOLO SERIE (115 200 BAUDIOS, LÍNEAS TERMINADAS EN LF)

Mensaje	Sentido	Significado
D,p,r,jx,jy,b	ESP32→PC	IMU (°), joystick (−1…1), botón
K,id	ESP32→PC	Respuesta al ping
V,1 / V,2 / V,3	PC→ESP32	Vibrar: objeto / pared / obstáculo
P,id	PC→ESP32	Ping de latencia
C	PC→ESP32	Recalibrar giroscopio y joystick
S	Monitor	Diagnóstico (IMU, ADC, bus I²C)

VI. Fase 2 – Desarrollo del entorno virtual

El proyecto se creó en Unity 6 con el *pipeline* integrado, espacio de color lineal y post-procesado. Toda la escena se genera por código con el script de editor `SceneBuilder` (menú *Teleoperación → Construir escena*), lo que garantiza que se pueda reconstruir de forma reproducible. Los pasos fueron:

A. Paso 1: diseño del escenario

Se diseñó una base de exploración en Marte dentro de una arena de 20 × 20 m (Fig. 3): terreno *low-poly* con dunas y montañas, textura de suelo marciano generada proceduralmente, cielo color caramelo con halo azulado alrededor del Sol (*shader* propio `MarsSky`), niebla de polvo, partículas ambientales y post-procesado (bloom, tonemapping ACES, oclusión ambiental y viñeta).

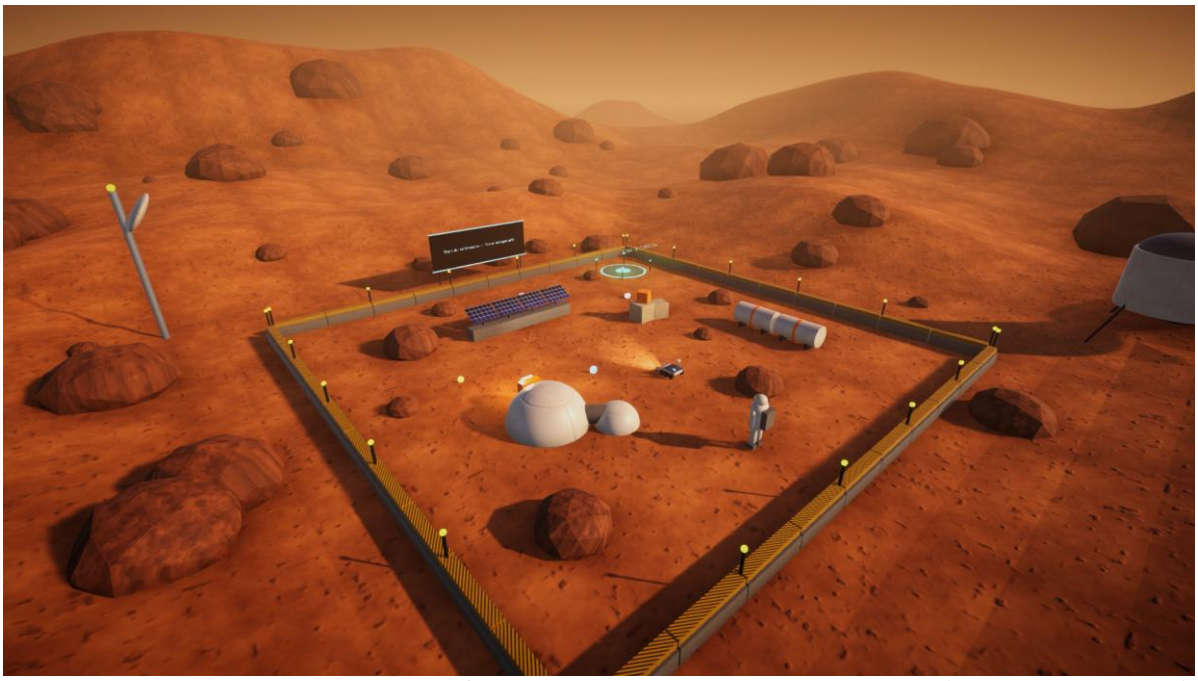


Fig. 3. Vista general del entorno virtual: arena con barreras, módulo habitacional, paneles solares, tanques, rocas, plataforma de entrega y panel informativo.

B. Paso 2: navegación y cámara

El punto de vista principal es una cámara que va ligada al robot (`ChaseCamera`) y cuya orientación alrededor de él se controla con la IMU (Fig. 4). Además, el escenario incluye un avatar astronauta con `CharacterController` y cámara a 1,65 m que permite la navegación en primera persona (tecla V); su cabeza se orienta con el visor y avanza o retrocede en la dirección de la mirada. La cámara evita atravesar paredes mediante un *SphereCast* desde el robot.



Fig. 4. Vista de la cámara que sigue al robot, controlada por la inclinación de la MPU6050.

C. Paso 3: límites físicos con colliders

El perímetro tiene cuatro `BoxCollider` de 1,2 m de altura y una segunda barrera invisible superior que impide que el robot, los objetos o el avatar salgan del área. Visualmente se representan con barreras de franjas de peligro y balizas pulsantes. Todos los obstáculos (módulo habitacional, cajas, bancada de paneles solares, tanques y ocho rocas con `MeshCollider` convexo) tienen collider sólido y un componente `HapticSurface` que define su patrón de vibración.

D. Paso 4: robot móvil con física

El robot es un *rover* de seis ruedas (Fig. 5) con `Rigidbody` de 8 kg, rotaciones X/Z bloqueadas, detección de colisiones continua y material sin fricción en el chasis. `RobotController` aplica en `FixedUpdate` la velocidad lineal y angular filtradas (Sección IX) y anima las ruedas con la cinemática diferencial $v_{l,r} = v \pm \omega\, b/2$. Incluye faros, polvo en las ruedas y chispas con sacudida de cámara al chocar.

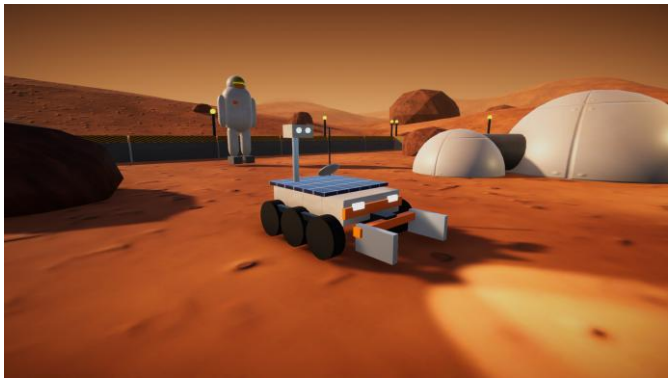


Fig. 5. Robot móvil virtual (rover) con panel solar, mástil con cámara, faros y garra frontal.

E. Paso 5: sistema de agarre (attach/detach)

La garra tiene una zona *trigger* entre los dedos. Al presionar el botón del joystick la garra se cierra; si hay un objeto manipulable en la zona, se crea una `FixedJoint` entre el objeto y el robot (*attach*) y se ignoran sus colisiones mutuas; al presionar de nuevo, la junta se destruye (*detach*). Los objetos son cuatro cristales luminosos que deben llevarse a la

plataforma de entrega (Fig. 6), donde `GoalZone` cuenta los depositados y `TeleopMetrics` mide el tiempo de la tarea.

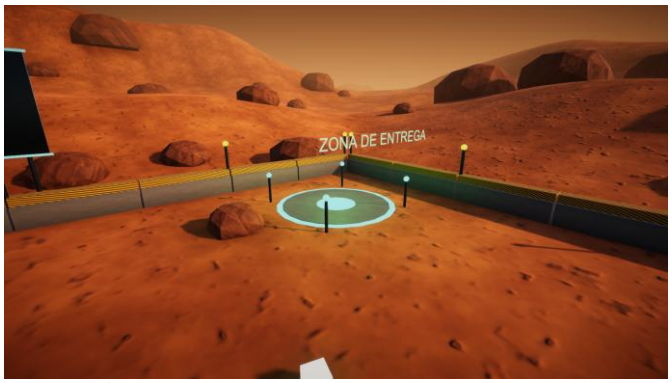


Fig. 6. Plataforma de entrega con anillo luminoso y balizas.

VII. Fase 3 – Desarrollo del control háptico

A. Selección de componentes y conexiones

Se usó un ESP32 DevKit V1 por integrar Bluetooth Clásico, ADC de 12 bits y PWM por hardware (LEDC) [11]. El joystick se conectó a pines del ADC1 (GPIO34/35), que siguen funcionando con la radio activa, y se alimentó a 3,3 V para aprovechar todo el rango del ADC. El motor se maneja con un transistor 2N2222, porque un GPIO no entrega la corriente del motor (≈ 80 mA), y lleva un diodo de rueda libre. Las conexiones se muestran en la Fig. 7 y la Tabla IV.

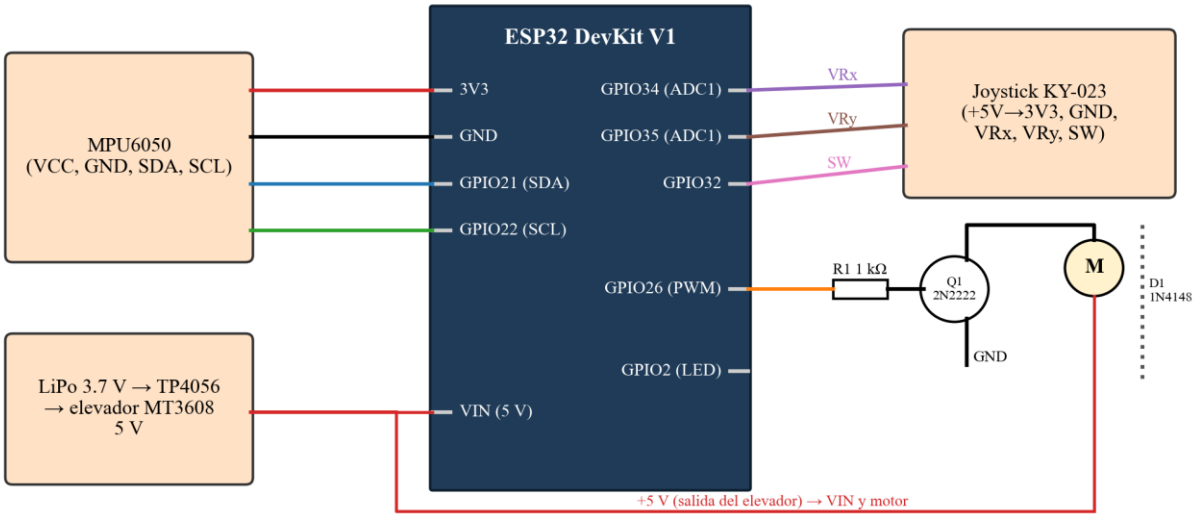


Fig. 7. Diagrama eléctrico del control háptico: ESP32, MPU6050, joystick, driver del motor vibrador y alimentación.

TABLA IV
ASIGNACIÓN DE PINES DEL ESP32

Módulo	Pin del módulo	ESP32
MPU6050	VCC / GND	3V3 / GND
MPU6050	SDA / SCL	GPIO21 / GPIO22
Joystick	+5V / GND	3V3 / GND
Joystick	VRx (giro)	GPIO34
Joystick	VRy (avance)	GPIO35
Joystick	SW (garra)	GPIO32 (pull-up)
Vibrador	Base Q1 vía 1 kΩ	GPIO26 (PWM)
LED placa	—	GPIO2
Alimentación	5 V del MT3608	VIN

B. Lectura y filtrado de la IMU

La MPU6050 se configuró a ±2 g y ±500 °/s con filtro digital paso bajo de 44 Hz. Al encender, el firmware detecta la dirección I²C (0x68 o 0x69), promedia 300 muestras del giroscopio con el control quieto para eliminar el sesgo y aplica el filtro complementario de la ecuación (2) en cada ciclo. Si la IMU no responde, reintenta cada 2 s.

C. Lectura del joystick

El centro de cada eje se calibra al encender. La lectura se normaliza a [−1, 1] por separado a cada lado del centro, lo que compensa la asimetría típica de los potenciómetros:

$$j = (x - c)/(4095 - c) \;\; si\; x \geq c; \;\; (x - c)/c \;\; si\; x < c \qquad (4)$$

D. Transmisión Bluetooth

El ESP32 se anuncia como **ControlHaptico_UMNG** (perfil SPP) y envía las mismas tramas por USB y por Bluetooth, lo que permitió depurar por cable y luego operar de forma inalámbrica. Cada 20 ms se transmite `D,pitch,roll,jx,jy,b` (Apéndice A).

E. Control PWM y patrones de vibración

El motor se controla con LEDC a 5 kHz y 8 bits. Los patrones se reproducen sin bloquear el programa mediante una tabla de pasos {ciclo útil, duración} (Tabla V), de modo que la lectura de sensores sigue a 50 Hz durante la vibración.

TABLA V
PATRONES DE VIBRACIÓN POR TIPO DE COLISIÓN

Colisión con	Patrón	Ciclo útil
Objeto manipulable (V,1)	2 toques de 70 ms	150/255 (59 %)
Pared o límite (V,2)	1 pulso de 400 ms	255/255 (100 %)
Obstáculo (V,3)	3 pulsos de 50 ms	210/255 (82 %)

F. Carga del firmware paso a paso

1) Instalar Arduino IDE 2 y, en el gestor de placas, el núcleo *esp32* de Espressif.
2) Abrir `Firmware/ESP32_ControlHaptico/ESP32_ControlHaptico.ino` y seleccionar la placa *ESP32 Dev Module* y el puerto COM del CP2102.
3) Si la carga falla por ruido, bajar *Upload Speed* a 115 200 (en las pruebas fue necesario).
4) Abrir el Monitor Serie a 115 200 baudios, enviar `S` y verificar que el bus I²C muestre la MPU6050 y que el joystick cambie de valor.
5) Emparejar el ESP32 en Windows (*Configuración → Bluetooth → Agregar dispositivo*); Windows crea un puerto COM de Bluetooth.

VIII. Fase 4 – Manufactura y Ensamble

La carcasa del control se diseñó en SOLIDWORKS como una empuñadura de dos piezas que aloja el ESP32, la batería y el cargador en el cuerpo, la MPU6050 fija y alineada con el eje longitudinal (para que pitch y roll correspondan a la muñeca), el joystick bajo el pulgar y el motor vibrador en contacto con la palma. Se imprimió en PLA por FDM.

[COMPLETAR: insertar render/plano del CAD de la carcasa]

Fig. 8. Diseño CAD de la carcasa del control háptico (SOLIDWORKS).

[COMPLETAR: insertar foto del prototipo físico]

Fig. 9. Control háptico impreso y ensamblado con la electrónica.

[COMPLETAR: parámetros de impresión — altura de capa, relleno, tiempo y masa reales]

IX. Fase 5 – Integración y Pruebas

A. Comunicación Unity–Bluetooth

`HapticLink` lee el puerto serie en un hilo dedicado para no bloquear el *render*. Para encontrar el control sin configuración, consulta el registro de Windows: asocia cada puerto COM de Bluetooth con la dirección MAC del dispositivo y su nombre, intenta primero el último puerto que funcionó, luego los USB y por último el puerto de **ControlHaptico_UMNG**, y descarta los de otros aparatos (audífonos, celulares). Abre el puerto con DTR/RTS desactivados para no reiniciar el ESP32 y valida la conexión al recibir una trama `D,`.

B. Mapeo matemático del control

Para el robot, cada eje del joystick pasa por una zona muerta j_m = 0,12 y una normalización lineal; la referencia resultante se filtra con un sistema de primer orden discreto para obtener una respuesta suave:

$$u(j) = \mathrm{sign}(j) \cdot \mathrm{clamp}((|j| - j_m)/(1 - j_m),\ 0,\ 1) \quad (5)$$

$$v^* = v_max\ u(j_y),\quad \omega^* = \omega_max\ u(j_x) \quad (6)$$

$$v_k = v_{k-1} + (v^* - v_{k-1})(1 - e^{(-\lambda \Delta t)}),\ \lambda = 6\ s^{-1} \quad (7)$$

Para la cámara, la IMU pasa por una zona muerta de 2° y define la elevación y el azimut alrededor del robot, de modo que la cámara siempre sigue la posición y el giro del robot:

$$\varepsilon = \mathrm{clamp}(22° + 1{,}2\,\theta,\ 3°,\ 80°) \quad (8)$$

$$\psi = \psi_robot + 3\,\varphi \quad (9)$$

$$p_cam = p_robot + R(\varepsilon, \psi)\,[0, 0, -2{,}2]^T\,m \quad (10)$$

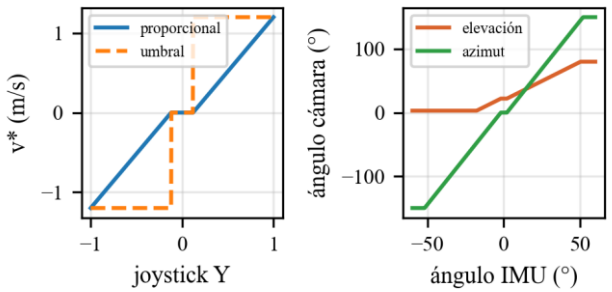


Fig. 10. Curvas de mapeo: velocidad del robot frente al joystick (proporcional y por umbral) y ángulos de la cámara frente a la inclinación de la IMU.

C. Visualización en el celular como gafas de RV

Unity Remote 5 no admite iPhone en Windows, por lo que se implementó `PhoneStreamer`: un servidor HTTP propio que renderiza la cámara del robot dos veces (ojo izquierdo y derecho), aplica la pre-distorsión de la ecuación (3) con el *shader* `CardboardDistortion` y envía el resultado como video MJPEG a ~30 fps (Fig. 12). En Safari se abre `http://IP-del-PC:8765`; la página permite ajustar IPD, distorsión, zoom y campo de visión, y se instala como aplicación a pantalla completa. Para ello se configuró la red como privada y se abrió el puerto 8765 en el Firewall.

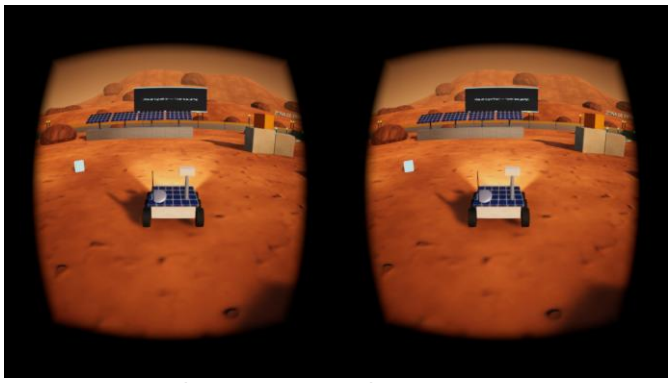


Fig. 11. Imagen estéreo con pre-distorsión de lentes que recibe el celular dentro del visor Cardboard.

D. Ajuste de sensibilidad

Los parámetros quedaron expuestos en el Inspector de Unity: zona muerta, velocidades máximas, constante λ, modo proporcional o por umbral, ganancias de la cámara e inversión de ejes. La tecla C (o el comando `c`) recalibra el cero de la IMU con la mano en reposo.

X. Resultados Experimentales

Cada sesión genera dos archivos CSV (eventos y telemetría a 20 Hz). Se analizaron las sesiones realizadas con el ESP32: 11 por USB y 3 por Bluetooth. con 126.477 mediciones de latencia.

A. Prueba de colisiones

Con el script de editor `TeleopDiagnostics` se lanzó el robot a velocidad máxima durante 3 s contra cada obstáculo y límite usando el motor de física real. En los 15 casos el robot se detuvo contra la superficie, con penetraciones de pocos centímetros, propias del margen de contacto de PhysX (Tabla VI).

TABLA VI
PRUEBA DE COLISIÓN DEL ROBOT (V = 1,2 M/S, 3 S)

Objeto	Libre (m)	Penetr. (cm)	Resultado
Roca 1	1.28	+1.3	Bloquea
Roca 7	1.30	-0.7	Bloquea
Límite Norte	1.73	-0.6	Bloquea
Límite Sur	1.73	-0.6	Bloquea
Límite Oeste	1.73	-0.6	Bloquea
Módulo habitacional	2.14	+5.2	Bloquea
Cajas	1.70	+5.1	Bloquea
Tanques	1.33	+1.5	Bloquea
Paneles solares	3.48	-0.6	Bloquea
Roca 6	1.30	+0.3	Bloquea
Roca 5	1.28	-0.4	Bloquea
Roca 4	1.28	+0.5	Bloquea
Roca 3	1.29	+0.7	Bloquea
Roca 8	1.30	-0.1	Bloquea
Roca 2	1.28	-0.4	Bloquea

B. Latencia de comunicación

TABLA VII
LATENCIA DE IDA Y VUELTA (RTT) MEDIDA CON PING

Estadístico	USB	Bluetooth SPP
Muestras	114.947	11.530
Mediana (ms)	2.9	46.1
Media (ms)	4.3	53.6
Percentil 95 (ms)	5.7	137.0
Percentil 99 (ms)	32.1	193.1
Máximo (ms)	877	929

La latencia por Bluetooth (mediana 46.1 ms, P95 137 ms) es unas 16 veces la del USB, pero sigue por debajo de los ~100 ms en que el retardo empieza a percibirse en la teleoperación [5]. Los máximos aislados (> 800 ms) corresponden a pausas del editor de Unity y no al enlace (Fig. 12). La tasa de recepción se mantuvo en 50 Hz.

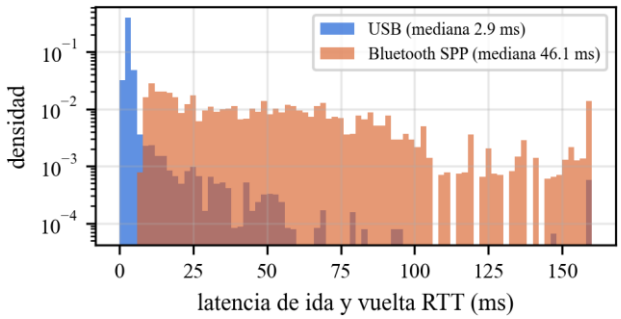


Fig. 12. Distribución de la latencia RTT por USB y por Bluetooth (escala logarítmica).

C. Respuesta dinámica del control

La Fig. 13 muestra un escalón real del joystick (de 0 a 1) tomado de la telemetría. La velocidad alcanzó el 63 % de su valor final en $\tau \approx 205$ ms, frente a $1/\lambda = 167$ ms del modelo; la diferencia se explica por el periodo de muestreo de la telemetría (50 ms) y la latencia del enlace. No se observaron oscilaciones ni sobrepicos, lo que confirma la estabilidad del mapeo.

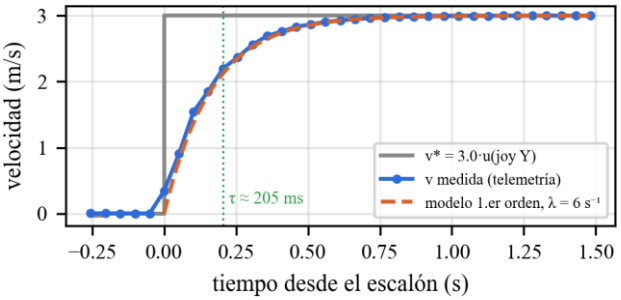


Fig. 13. Respuesta al escalón del joystick: referencia, velocidad medida y modelo de primer orden.

D. Sesiones de uso

En 12 sesiones con el ESP32 se registraron 197 eventos de vibración: 89 con obstáculos, 71 con límites y 37 con objetos manipulables (Fig. 14), además de 16 agarres. El mejor desempeño fue 3 de 4 muestras entregadas. La Fig. 15 muestra las señales de la IMU, del joystick y la velocidad del robot durante una sesión de 5.9 min por Bluetooth.

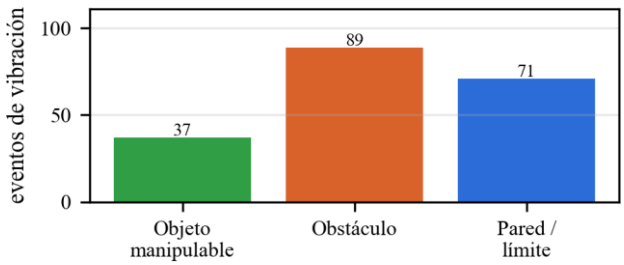


Fig. 14. Eventos de vibración por tipo de colisión en las sesiones con el ESP32.

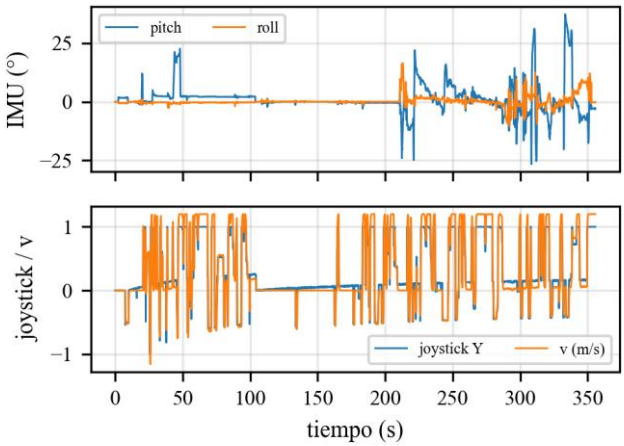


Fig. 15. Telemetría de una sesión por Bluetooth: inclinación de la IMU (arriba) y joystick con velocidad del robot (abajo).

Video demostrativo: **[COMPLETAR: enlace de YouTube]**

XI. Análisis de resultados

A. Implementación del sistema

Se cumplieron los requerimientos de la guía: entorno con obstáculos, objetos manipulables y límites; robot con avance, giro, detección de colisiones y agarre; control inalámbrico con IMU, joystick, botón, actuador PWM y batería, y visualización inmersiva en el celular. La prueba automática de colisiones (15/15) y los registros de agarre y entrega confirman el funcionamiento integrado.

B. Calidad percibida de la retroalimentación háptica

Los tres patrones difieren a la vez en intensidad, duración y ritmo, lo que facilita distinguirlos sin mirar la pantalla. El enfriamiento de 0,35 s entre vibraciones evita que el contacto sostenido con una pared sature al usuario. **[COMPLETAR: resultados de la encuesta a usuarios — porcentaje de patrones identificados correctamente y calificación 1–5]**

C. Estabilidad del control

La zona muerta del joystick eliminó el movimiento espontáneo por el ruido del centro, y el filtro de primer orden produjo una respuesta sin sobrepico. En la cámara, el filtro complementario y la zona muerta de 2° evitaron el temblor con la mano quieta; el suavizado exponencial ($k = 8\ s^{-1}$) mantuvo la cámara ligada al robot sin saltos. El aumento de latencia por Bluetooth fue tolerable para la tarea.

D. Facilidad de uso

Separar el control del robot (pulgar) y el de la cámara (inclinación de la mano) resultó intuitivo, y la detección automática del puerto simplifica la puesta en marcha. Como limitaciones se identificaron la dependencia de la red Wi-Fi para el video y la necesidad de recalibrar la IMU si se cambia el agarre del control.

XII. Preguntas para la discusión

¿Cómo afecta la latencia del Bluetooth a la percepción de control? Con una mediana de 46 ms el retardo no se percibe al conducir, pero los picos de más de 130 ms (P95) generan una leve sensación de inercia en los giros rápidos. Por encima de ~200 ms el operador tiende a sobrecorregir y aparecen oscilaciones.

¿Qué filtro es más adecuado para estabilizar la IMU? Para pitch y roll el filtro complementario es suficiente y económico en cómputo. Si se necesitara además el yaw sin deriva, o mayor robustez ante aceleraciones, convendría un filtro de Kalman o de Mahony/Madgwick con magnetómetro [7].

¿Qué ventajas tiene el control proporcional frente al de umbrales? Permite velocidades intermedias para maniobras finas, como alinear la garra con un objeto, y produce menos choques. El control por umbral es más sencillo, pero obliga a moverse siempre a la velocidad máxima; el sistema incluye ambos modos para compararlos.

¿Cómo cambiaría el diseño si el robot fuera físico? Haría falta odometría y control de velocidad en los motores (PID), paro de emergencia y temporizador de seguridad ante pérdida de enlace, y realimentación de sensores reales (distancia, corriente) para generar la háptica. Además habría que transmitir video real con baja latencia.

¿Qué variables influyen en la percepción diferenciada de vibraciones? La amplitud (ciclo útil), la frecuencia de vibración del motor, la duración y el número de pulsos, el ritmo, la ubicación del actuador y el acoplamiento mecánico con la mano.

¿Cómo implementar una háptica más rica (fuerza)? Con actuadores de retroalimentación de fuerza: joystick con motores (*force feedback*), frenos magnetorreológicos o actuadores lineales resonantes (LRA) con varias frecuencias, de modo que el usuario sienta resistencia proporcional a la fuerza de contacto calculada por la física.

¿Qué implicaciones tiene en aplicaciones médicas o industriales? Permite entrenar a operarios y cirujanos sin riesgo, teleoperar en ambientes peligrosos y mejorar la precisión gracias a la retroalimentación. En esos casos exige certificar la latencia, la seguridad del enlace y la confiabilidad del sistema.

¿Cómo mejorar la inmersión? Usar el giroscopio del visor para la cabeza (HTTPS o una aplicación nativa), un visor autónomo como el Meta Quest 3, audio espacial, mayor tasa de cuadros, y háptica de fuerza en lugar de solo vibración.

XIII. Conclusiones

Se integró un sistema de teleoperación inmersivo funcional que combina programación de entornos virtuales, diseño mecatrónico, comunicaciones inalámbricas y háptica. La separación entre el control del robot (joystick) y el de la cámara (IMU) resultó un mapeo natural, y el filtrado (complementario en la IMU, zona muerta y primer orden en el robot) fue determinante para la estabilidad.

El enlace Bluetooth SPP ofreció una latencia mediana de 46 ms, adecuada para la tarea aunque mayor que la del USB (2.9 ms), y la retroalimentación vibrotáctil diferenciada por PWM permitió reconocer el tipo de colisión sin depender de la vista. La transmisión estéreo con pre-distorsión convirtió un celular en un visor de RV de bajo costo. Como trabajo futuro se propone el seguimiento de cabeza con el giroscopio del visor, háptica de fuerza y la migración a un robot físico.

Referencias

[1] S. M. LaValle, *Virtual Reality*. Cambridge, U.K.: Cambridge Univ. Press, 2023.

[2] W. R. Sherman and A. B. Craig, *Understanding Virtual Reality: Interface, Application, and Design*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2018.

[3] G. C. Burdea and P. Coiffet, *Virtual Reality Technology*, 2nd ed. Hoboken, NJ, USA: Wiley, 2003.

[4] T. B. Sheridan, *Telerobotics, Automation, and Human Supervisory Control*. Cambridge, MA, USA: MIT Press, 1992.

[5] J. Y. C. Chen, E. C. Haas, and M. J. Barnes, "Human performance issues and user interface design for teleoperated robots," *IEEE Trans. Syst., Man, Cybern. C*, vol. 37, no. 6, pp. 1231–1245, Nov. 2007.

[6] InvenSense, "MPU-6000 and MPU-6050 Product Specification," Rev. 3.4, 2013.

[7] R. Mahony, T. Hamel, and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Trans. Autom. Control*, vol. 53, no. 5, pp. 1203–1218, Jun. 2008.

[8] S. Choi and K. J. Kuchenbecker, "Vibrotactile display: Perception, technology, and applications," *Proc. IEEE*, vol. 101, no. 9, pp. 2093–2104, Sep. 2013.

[9] Bluetooth SIG, "Serial Port Profile (SPP)," Specification v1.2, 2012.

[10] Google, "Google Cardboard: Viewer profile and lens distortion," Google for Developers. [En línea]. Disponible: https://developers.google.com/cardboard

[11] Espressif Systems, "ESP32 Series Datasheet," v4.x, 2023.

[12] Unity Technologies, "Unity 6 Manual: Physics and Rigidbody," 2025. [En línea]. Disponible: https://docs.unity3d.com/Manual

Apéndice A. Firmware del control háptico (ESP32)

Archivo `Firmware/ESP32_ControlHaptico/ESP32_ControlHaptico.ino`. Lee la MPU6050 y el joystick, transmite por USB y Bluetooth, responde pings y reproduce los patrones PWM.

```
/*
  Control háptico – Teleoperación de robot móvil en entorno virtual (UMNG, Realidad Virtual)

  Funciones:
    - MPU6050 (inclinación pitch / roll)  -> mueve la CÁMARA que sigue al carro en Unity
    - Joystick analógico (KY-023)          -> mueve el CARRO (Y = avance/retroceso, X = giro)
    - Botón del joystick (presionar)        -> cierra / abre la GARRA
    - Motor vibrador por PWM                -> vibra cuando el carro choca (patró
    n distinto según el objeto)

  ======================== CONEXIONES (ESP32 DevKit, 3.3 V) ========================
    MPU6050      VCC -> 3V3      GND -> GND      SDA -> GPIO21     SCL -> GPIO22
    Joystick     +5V -> 3V3 (¡no a 5 V!)  GND -> GND
                 VRx -> GPIO34   VRy -> GPIO35   SW -> GPIO32  (pull-up interno)
    Vibrador     GPIO26 -> resistencia 1 kΩ -> base transistor NPN 2N2222 (o S8050)
                 emisor -> GND ; colector -> (–) del motor ; (+) del motor -> 3V3 o 5V(VIN)
                 diodo 1N4148 en paralelo al motor (cátodo hacia +)
                 * Si usas un MÓDULO vibrador con driver: IN -> GPIO26, VCC -> 3V3, GND -> GND
    LED de la placa: GPIO2 (parpadea al calibrar, encendido con cliente Bluetooth)
  ==================================================================================

  Al encender: dejar el control QUIETO y el joystick SIN TOCAR ~1 s (calibra giroscopio y centro del joystick).

  Protocolo con Unity (texto, 115200 baudios, por USB y también por Bluetooth "ControlHaptico_UMNG"):
    ESP32 -> PC : D,<pitch>,<roll>,<joyX>,<joyY>,<boton>     50 Hz  (ángulos en °, joystick -1.00..1.00, botón 0/1)
                  K,<id>                                     respuesta a ping (latencia)
    PC -> ESP32 : V,<patrón>   1 = objeto manipulable, 2 = pared/límite, 3 = obstáculo
                  P,<id>       ping

  Placa: "ESP32 Dev Module". Núcleo Arduino-ESP32 2.x o 3.x. Sin librerías externas.
*/

#include <Wire.h>

// Bluetooth Classic solo existe en el ESP32 original (no en S2/S3/C3). Si no está, funciona solo por USB.
#if defined(CONFIG_BT_ENABLED) && defined(CONFIG_BLUEDROID_ENABLED) && !defined(CONFIG_IDF_TARGET_ESP32S3) && !defined(CONFIG_IDF_TARGET_ESP32C3) &&
!defined(CONFIG_IDF_TARGET_ESP32S2)
#define USE_BT 1
#include "BluetoothSerial.h"
BluetoothSerial SerialBT;
#else
#define USE_BT 0
#endif

// ---------------- Pines
const int PIN_SDA = 21, PIN_SCL = 22;
const int PIN_JOY_X = 34, PIN_JOY_Y = 35, PIN_JOY_SW = 32;
const int PIN_MOTOR = 26;
const int PIN_LED = 2;
```

```
// ---------------- Ajustes
const bool INVERT_JOY_X = false;   // cambia a true si el giro sale al revés
const bool INVERT_JOY_Y = true;    // en el KY-023, empujar hacia delante BAJA el valor -> se invierte
const float ALPHA = 0.98f;         // filtro complementario de la IMU
const unsigned long SEND_PERIOD_MS = 20;   // 50 Hz


// ---------------- PWM del vibrador
const int PWM_FREQ = 5000;
const int PWM_RES = 8;   // duty 0..255
const int PWM_CH = 0;    // solo núcleo 2.x


// ---------------- Estado IMU
uint8_t MPU = 0x68;   // 0x69 si el pin AD0 del módulo está en alto (se detecta solo)
bool imuOk = false;
float gyroBiasX = 0, gyroBiasY = 0;
float pitch = 0, roll = 0;
unsigned long lastImuMicros = 0;


// ---------------- Estado joystick
int joyCenterX = 2048, joyCenterY = 2048;
float joyX = 0, joyY = 0;


unsigned long lastSend = 0;
String rxUsb, rxBt;


// ---------------- Patrones de vibración {duty 0..255, ms}, terminan en {0,0}
struct Step { uint8_t duty; uint16_t ms; };
const Step PAT_OBJ[]  = { {150, 70}, {0, 60}, {150, 70}, {0, 0} };                       // 2 toques suaves
const Step PAT_WALL[] = { {255, 400}, {0, 0} };                                          // 1 golpe largo e intenso
const Step PAT_OBS[]  = { {210, 50}, {0, 40}, {210, 50}, {0, 40}, {210, 50}, {0, 0} };   // 3 pulsos medios
const Step* curPattern = nullptr;
int curStep = 0;
unsigned long stepStart = 0;


// ============================================================ vibración


void motorWrite(uint8_t duty) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(PIN_MOTOR, duty);
#else
  ledcWrite(PWM_CH, duty);
#endif
}


void startPattern(int n) {
  switch (n) {
    case 1: curPattern = PAT_OBJ; break;
    case 2: curPattern = PAT_WALL; break;
    case 3: curPattern = PAT_OBS; break;
    default: return;
  }
  curStep = 0;
  stepStart = millis();
  motorWrite(curPattern[0].duty);
}


void updatePattern() {
  if (!curPattern) return;
  if (millis() - stepStart < curPattern[curStep].ms) return;
  curStep++;
  stepStart = millis();
  if (curPattern[curStep].ms == 0) { motorWrite(0); curPattern = nullptr; return; }
  motorWrite(curPattern[curStep].duty);
}


// ============================================================ MPU6050


void mpuWrite(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(MPU); Wire.write(reg); Wire.write(val); Wire.endTransmission();
}


bool mpuRead(int16_t& ax, int16_t& ay, int16_t& az, int16_t& gx, int16_t& gy, int16_t& gz) {
  Wire.beginTransmission(MPU);
  Wire.write(0x3B);
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(MPU, (uint8_t)14) != 14) return false;
  ax = Wire.read() << 8 | Wire.read();
  ay = Wire.read() << 8 | Wire.read();
  az = Wire.read() << 8 | Wire.read();
  Wire.read(); Wire.read();   // temperatura
  gx = Wire.read() << 8 | Wire.read();
  gy = Wire.read() << 8 | Wire.read();
  gz = Wire.read() << 8 | Wire.read();
  return true;
}
```

```
bool initImu() {
  imuOk = false;
  for (uint8_t a : { (uint8_t)0x68, (uint8_t)0x69 }) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) { MPU = a; imuOk = true; break; }
  }
  if (imuOk) {
    mpuWrite(0x6B, 0x00);   // despertar
    mpuWrite(0x1A, 0x03);   // filtro DLPF ~44 Hz
    mpuWrite(0x1B, 0x08);   // giroscopio ±500 °/s
    mpuWrite(0x1C, 0x00);   // acelerómetro ±2 g
    delay(100);
  }
  return imuOk;
}

void accelAngles(int16_t ax, int16_t ay, int16_t az, float& p, float& r) {
  float fax = ax / 16384.0f, fay = ay / 16384.0f, faz = az / 16384.0f;   // ±2 g
  p = atan2f(-fax, sqrtf(fay * fay + faz * faz)) * RAD_TO_DEG;
  r = atan2f(fay, faz) * RAD_TO_DEG;
}

void updateImu() {
  if (!imuOk) return;
  int16_t ax, ay, az, gx, gy, gz;
  if (!mpuRead(ax, ay, az, gx, gy, gz)) return;
  unsigned long now = micros();
  float dt = (now - lastImuMicros) * 1e-6f;
  lastImuMicros = now;
  if (dt <= 0 || dt > 0.1f) dt = 0.005f;


  float pa, ra;
  accelAngles(ax, ay, az, pa, ra);
  float gxd = (gx - gyroBiasX) / 65.5f;   // ±500 °/s
  float gyd = (gy - gyroBiasY) / 65.5f;


  // Filtro complementario: giroscopio (rápido) + acelerómetro (sin deriva)
  pitch = ALPHA * (pitch + gyd * dt) + (1 - ALPHA) * pa;
  roll  = ALPHA * (roll  + gxd * dt) + (1 - ALPHA) * ra;
}

// ============================================================ joystick

float readAxis(int pin, int center, bool invert) {
  int raw = analogRead(pin);
  float v = (raw >= center) ? (raw - center) / float(4095 - center) : (raw - center) / float(center);
  v = constrain(v, -1.0f, 1.0f);
  return invert ? -v : v;
}

void calibrate() {
  // Giroscopio y centro del joystick: control quieto, joystick suelto
  long sgx = 0, sgy = 0, sjx = 0, sjy = 0; int ng = 0;
  int16_t ax, ay, az, gx, gy, gz;
  for (int i = 0; i < 300; i++) {
    if (imuOk && mpuRead(ax, ay, az, gx, gy, gz)) { sgx += gx; sgy += gy; ng++; }
    sjx += analogRead(PIN_JOY_X);
    sjy += analogRead(PIN_JOY_Y);
    digitalWrite(PIN_LED, (i / 30) % 2);
    delay(3);
  }
  if (ng) { gyroBiasX = sgx / (float)ng; gyroBiasY = sgy / (float)ng; }
  joyCenterX = sjx / 300;
  joyCenterY = sjy / 300;
  if (imuOk && mpuRead(ax, ay, az, gx, gy, gz)) accelAngles(ax, ay, az, pitch, roll);
  lastImuMicros = micros();
  digitalWrite(PIN_LED, LOW);
}

// ============================================================ comandos de Unity

void handleLine(const String& l, Stream& out) {
  if (l.startsWith("V,")) startPattern(l.substring(2).toInt());
  else if (l.startsWith("P,")) { out.print("K,"); out.print(l.substring(2)); out.print('\n'); }
  else if (l == "C") calibrate();
  else if (l == "S") printStatus(out);
}

// Diagnóstico: escribe "S" en el Monitor Serie (115200, salto de línea "LF")
void printStatus(Stream& out) {
  out.printf("# IMU %s (0x%02X)  pitch=%.1f roll=%.1f\n", imuOk ? "OK" : "NO RESPONDE", MPU, pitch, roll);
  out.printf("# Joystick crudo X=%d Y=%d (centro %d/%d)  SW=%d\n",
             analogRead(PIN_JOY_X), analogRead(PIN_JOY_Y), joyCenterX, joyCenterY, digitalRead(PIN_JOY_SW) == LOW);
  out.print("# I2C encontrados:");
  int n = 0;
  for (uint8_t a = 1; a < 127; a++) {
```

```
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) { out.printf(" 0x%02X", a); n++; }
  }
  out.println(n ? "" : " ninguno (revisa cables SDA/SCL y alimentacion)");
}


void readStream(Stream& s, String& line) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n') { line.trim(); handleLine(line, s); line = ""; }
    else if (line.length() < 32) line += c;
  }
}

// ============================================================ setup / loop

void setup() {
  Serial.begin(115200);
  pinMode(PIN_JOY_SW, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);   // rango completo 0..3.3 V


#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(PIN_MOTOR, PWM_FREQ, PWM_RES);
#else
  ledcSetup(PWM_CH, PWM_FREQ, PWM_RES);
  ledcAttachPin(PIN_MOTOR, PWM_CH);
#endif
  motorWrite(0);


  Wire.begin(PIN_SDA, PIN_SCL);
  Wire.setClock(400000);
  initImu();
  calibrate();


#if USE_BT
  SerialBT.begin("ControlHaptico_UMNG");
#endif
  Serial.println(imuOk ? "# MPU6050 OK" : "# ERROR: MPU6050 no responde (revisa SDA=21, SCL=22, 3V3, GND)");
  startPattern(1);   // vibración de bienvenida
}


unsigned long lastImuRetry = 0;


void loop() {
  // Si la IMU no respondió al encender, reintenta cada 2 s (por si se conecta después)
  if (!imuOk && millis() - lastImuRetry > 2000) {
    lastImuRetry = millis();
    if (initImu()) { calibrate(); Serial.println("# MPU6050 detectada"); }
  }
  updateImu();
  updatePattern();
  readStream(Serial, rxUsb);
#if USE_BT
  readStream(SerialBT, rxBt);
#endif


  unsigned long now = millis();
  if (now - lastSend >= SEND_PERIOD_MS) {
    lastSend = now;
    joyX = readAxis(PIN_JOY_X, joyCenterX, INVERT_JOY_X);
    joyY = readAxis(PIN_JOY_Y, joyCenterY, INVERT_JOY_Y);
    int sw = digitalRead(PIN_JOY_SW) == LOW;


    char buf[64];
    snprintf(buf, sizeof(buf), "D,%.2f,%.2f,%.2f,%.2f,%d\n", pitch, roll, joyX, joyY, sw);
    Serial.print(buf);
#if USE_BT
    if (SerialBT.hasClient()) SerialBT.print(buf);
    digitalWrite(PIN_LED, SerialBT.hasClient());
#endif
  }
  delay(2);
}
```

APÉNDICE B. CÓDIGOS PRINCIPALES EN UNITY (C#)

B.1 Recepción de tramas y latencia (HapticLink.cs)

```
    void ParseLine(string line)
    {
        if (line.Length < 2 || line[0] == '#') return;
        var parts = line.Split(',');
        var inv = CultureInfo.InvariantCulture;
        double now = stopwatch.Elapsed.TotalMilliseconds;
```

```
    if (parts[0] == "D" && parts.Length >= 6)
    {
        if (!float.TryParse(parts[1], NumberStyles.Float, inv, out float p)) return;
        if (!float.TryParse(parts[2], NumberStyles.Float, inv, out float r)) return;
        if (!float.TryParse(parts[3], NumberStyles.Float, inv, out float jx)) return;
        if (!float.TryParse(parts[4], NumberStyles.Float, inv, out float jy)) return;
        lock (sync)
        {
            rawPitch = p;
            rawRoll = r;
            rawJoyX = Mathf.Clamp(jx, -1f, 1f);
            rawJoyY = Mathf.Clamp(jy, -1f, 1f);
            rawButton = parts[5] == "1";
        }
        lastDataTime = now / 1000.0;
        Status = "Conectado al ESP32 en " + ActivePort;


        linesThisSecond++;
        if (now - rateWindowStart >= 1000)
        {
            DataRateHz = linesThisSecond * 1000f / (float)(now - rateWindowStart);
            linesThisSecond = 0;
            rateWindowStart = now;
        }
    }
    else if (parts[0] == "K" && parts.Length >= 2 && int.TryParse(parts[1], out int id))
    {
        lock (sync)
        {
            if (pendingPings.TryGetValue(id, out double sent))
            {
                pendingPings.Remove(id);
                float rtt = (float)(now - sent);
                LastLatencyMs = rtt;
                LatencySamples++;
                latencySum += rtt;
                AvgLatencyMs = latencySum / LatencySamples;
                MaxLatencyMs = Mathf.Max(MaxLatencyMs, rtt);
            }
        }
    }
}
```

B.2 Envío de vibración (HapticLink.cs)

```
public void Vibrate(HapticPattern pattern)
{
    WriteLine("V," + (int)pattern);
    Vibrated?.Invoke(pattern);
}
```

B.3 Mapeo y física del robot (RobotController.cs)

```
public float Map(float j)
{
    float a = Mathf.Abs(j);
    if (a < joystickDeadZone) return 0f;
    if (mode == MappingMode.Umbral) return Mathf.Sign(j);
    return Mathf.Sign(j) * Mathf.Clamp01((a - joystickDeadZone) / (1f - joystickDeadZone));
}


void FixedUpdate()
{
    var link = HapticLink.Instance;
    float jy = link ? link.DriveY : 0f;
    float jx = link ? link.DriveX : 0f;
    if (invertForward) jy = -jy;
    if (invertTurn) jx = -jx;


    float targetV = maxLinearSpeed * Map(jy);
    float targetW = maxAngularSpeed * Map(jx);


    float k = 1f - Mathf.Exp(-smoothing * Time.fixedDeltaTime);
    LinearSpeed += (targetV - LinearSpeed) * k;
    AngularSpeed += (targetW - AngularSpeed) * k;
    if (Mathf.Abs(targetV) > 0.01f || Mathf.Abs(targetW) > 0.01f) HasMoved = true;


    Vector3 vel = transform.forward * LinearSpeed;
    vel.y = rb.linearVelocity.y;
    rb.linearVelocity = vel;
    rb.angularVelocity = Vector3.up * (AngularSpeed * Mathf.Deg2Rad);
}


void OnCollisionEnter(Collision c)
{
    var surface = c.collider.GetComponentInParent<HapticSurface>();
    if (surface == null || Time.time < nextHapticTime) return;
```

```
        nextHapticTime = Time.time + hapticCooldown;
        if (HapticLink.Instance) HapticLink.Instance.Vibrate(surface.pattern);
        if (TeleopMetrics.Instance) TeleopMetrics.Instance.RegisterCollision(surface.pattern, c.collider.name);
    }
```

B.4 Cámara ligada al robot y controlada por la IMU (ChaseCamera.cs)

```
    void LateUpdate()
    {
        if (!target || !cam.enabled) return;


        var link = HapticLink.Instance;
        float p = link ? Dead(link.Pitch, imuDeadZone) : 0f;
        float r = link ? Dead(link.Roll, imuDeadZone) : 0f;
        if (invertPitch) p = -p;
        if (invertRoll) r = -r;


        float k = 1f - Mathf.Exp(-followSharpness * Time.deltaTime);
        float targetElev = Mathf.Clamp(baseElevation + pitchGain * p, minElevation, maxElevation);
        float targetOrbit = Mathf.Clamp(rollGain * r, -maxOrbit, maxOrbit);
        elevation = Mathf.Lerp(elevation, targetElev, k);
        orbit = Mathf.Lerp(orbit, targetOrbit, k);


        Vector3 focus = target.position + Vector3.up * targetHeight;
        float yaw = target.eulerAngles.y + orbit;   // ligada al giro del carro
        Quaternion rot = Quaternion.Euler(elevation, yaw, 0f);
        Vector3 desired = focus + rot * new Vector3(0f, 0f, -distance);


        // Evita atravesar paredes / obstáculos (ignora triggers y al propio carro)
        Vector3 dir = desired - focus;
        float d = dir.magnitude;
        foreach (var hit in Physics.SphereCastAll(focus, collisionRadius, dir / d, d, ~0, QueryTriggerInteraction.Ignore))
        {
            if (hit.rigidbody && hit.rigidbody.transform == target) continue;
            if (hit.transform.IsChildOf(target)) continue;
            if (hit.distance < d) d = Mathf.Max(0.3f, hit.distance);
        }
        desired = focus + dir.normalized * d;


        // Posición casi rígida respecto al carro (va ligada a él); el primer cuadro salta directo
        transform.position = placed ? Vector3.Lerp(transform.position, desired, Mathf.Max(k, 0.35f)) : desired;
        placed = true;
        transform.rotation = Quaternion.LookRotation(focus - transform.position, Vector3.up);


        // Sacudida al chocar (decae rápido)
        if (shake > 0.0005f)
        {
            float t = Time.time * 40f;
            transform.position += (transform.right * (Mathf.PerlinNoise(t, 0f) - 0.5f) +
                                   transform.up * (Mathf.PerlinNoise(0f, t) - 0.5f)) * (2f * shake);
            shake *= Mathf.Exp(-9f * Time.deltaTime);
        }
    }
```

B.5 Garra: agarre y liberación (RobotGripper.cs)

```
    void Update()
    {
        bool g = HapticLink.Instance && HapticLink.Instance.Grab;


        // Botón del joystick: la garra se cierra (y agarra el objeto que esté entre los dedos)
        if (grabMode == GrabMode.Alternar)
        {
            if (g && !prevGrab) Closed = !Closed;
        }
        else Closed = g;
        prevGrab = g;


        if (Closed && !Held) TryGrab();   // también agarra si el objeto entra con la garra cerrada
        else if (!Closed && Held) Release();


        // Por si el objeto se soltó por otra razón (joint roto, objeto destruido)
        if (Held && joint == null) Release();


        // Dedos: abiertos, cerrados sobre el objeto, o cerrados del todo si no hay objeto
        float target = !Closed ? fingerOpen : (Held ? fingerClosed : fingerEmpty);
        if (leftFinger) MoveFinger(leftFinger, -target);
        if (rightFinger) MoveFinger(rightFinger, target);
    }


    void TryGrab()
    {
        inZone.RemoveWhere(r => r == null);
        Rigidbody best = null;
        float bestD = float.MaxValue;
```

```
        foreach (var r in inZone)
        {
            float d = (r.worldCenterOfMass - transform.position).sqrMagnitude;
            if (d < bestD) { bestD = d; best = r; }
        }
        if (best == null) return;


        Held = best;
        AllHeld.Add(best);
        SetIgnore(best, true);
        joint = best.gameObject.AddComponent<FixedJoint>();
        joint.connectedBody = robotBody;
        joint.enablePreprocessing = false;
        if (TeleopMetrics.Instance) TeleopMetrics.Instance.RegisterEvent("agarre", best.name);
    }

    void Release()
    {
        if (joint) Destroy(joint);
        joint = null;
        if (Held)
        {
            SetIgnore(Held, false);
            AllHeld.Remove(Held);
            if (TeleopMetrics.Instance) TeleopMetrics.Instance.RegisterEvent("soltar", Held.name);
        }
        Held = null;
    }
```

B.6 Pre-distorsión de lentes del visor (CardboardDistortion.shader)

```
            fixed4 frag (v2f_img i) : SV_Target
            {
                bool right = i.uv.x >= 0.5;
                float2 uv = float2(right ? i.uv.x * 2 - 1 : i.uv.x * 2, i.uv.y);


                // Coordenadas centradas en el lente: vertical en [-1, 1], horizontal escalada por el aspecto
                float2 p = (uv - 0.5) * 2.0;
                p.x *= _EyeAspect;
                float r2 = dot(p, p);
                float2 q = p * (1.0 + _K1 * r2 + _K2 * r2 * r2) / _Zoom;
                q.x /= _EyeAspect;
                float2 src = q * 0.5 + 0.5;


                if (src.x < 0 || src.x > 1 || src.y < 0 || src.y > 1) return fixed4(0, 0, 0, 1);
                fixed4 c = right ? tex2D(_RightTex, src) : tex2D(_LeftTex, src);


                // Viñeta suave en el borde del lente
                float edge = saturate(1.0 - smoothstep(0.85, 1.0, max(abs(src.x - 0.5), abs(src.y - 0.5)) * 2.0));
                return fixed4(c.rgb * edge, 1);
            }
```

B.7 Superficies hápticas (HapticSurface.cs)

```
using UnityEngine;


/// Marca un objeto del entorno con el patrón de vibración que produce al chocar con el robot.
public class HapticSurface : MonoBehaviour
{
    public HapticPattern pattern = HapticPattern.ParedLimite;


    public bool IsManipulable => pattern == HapticPattern.ObjetoManipulable;
}
```

APÉNDICE C. ESTRUCTURA DEL PROYECTO Y EJECUCIÓN

```
TeleoperacionVR/
├── TeleoperacionRobotVR/                Proyecto Unity 6 (6000.6.2f1)
│   ├── Assets/Scenes/Teleoperacion.unity Escena principal
│   ├── Assets/Scripts/                  HapticLink, RobotController, RobotGripper, ChaseCamera,
│   │                                    AvatarController, PhoneStreamer, TeleopMetrics, GoalZone, RoverFX…
│   ├── Assets/Shaders/                  MarsSky.shader, CardboardDistortion.shader
│   ├── Assets/Editor/                   SceneBuilder, MarsAssets, TeleopDiagnostics, XRSetup
│   └── Resultados/                      CSV de eventos y telemetría por sesión
├── Firmware/ESP32_ControlHaptico/       Firmware del control
├── Capturas/                            Capturas y prueba_colisiones.csv
└── Informe/                             Este informe y sus figuras
```

1) Cargar el firmware al ESP32 y emparejarlo por Bluetooth como *ControlHaptico_UMNG*.
2) Abrir `TeleoperacionRobotVR` en Unity Hub y la escena `Assets/Scenes/Teleoperacion.unity`.
3) Pulsar Play: el HUD muestra el puerto, 50 Hz, la latencia y la dirección para el celular.

4) En el iPhone (misma red Wi-Fi) abrir en Safari la dirección del HUD, tocar *Gafas VR* y colocarlo en el visor.
5) Calibrar con la mano en reposo (tecla C), conducir con el joystick, orientar la cámara con la inclinación y cerrar la garra con el botón.